# A Stride-Based Convolution Decomposition Method to Stretch CNN Acceleration Algorithms for Efficient and Flexible Hardware Implementation

Chen Yang, Yizhou Wang, Xiaoli Wang, and Li Geng

***Abstract*— To reduce multiplication operations in convolution of convolutional neural networks (CNNs), there are three widely used convolutional acceleration algorithms, i.e., Winograd, FFT and FFA. However, current accelerators based on these convolutional acceleration algorithms have issues on flexibility and efficiency. Firstly, some accelerators utilized a combination of these acceleration algorithms and employed multiple types of computational units to achieve their respective advantages. As a result, some computational units are left unused when the best-performing unit is working, which causes much area inefficiency. Secondly, current accelerators tend to choose small parameters of these convolutional acceleration algorithms to avoid unacceptable precision loss, as a result, they are hardly to support large kernel sizes and lack of flexibility. Thirdly, these acceleration algorithms are typically presented for 1-stride convolutions, consequently, few implementation considers the acceleration of large-stride convolutions, which is a major restriction to hardware flexibility. This paper proposed a stride-based convolution decomposition method (SCDM) to reform different convolution shapes (i.e., kernel sizes & strides) to an identical pattern. With the aid of SCDM, a Winograd-stretched and hardware-efficient design (WHD) is presented to utilize one uniform computational unit for the acceleration of different convolution shapes, which combines complementary performance advantages on both Winograd *F(4,3)*and*F(4,2)*units. Compared to current FFT-based or FFA-based works, WHD can stretch the use range of Winograd and simplify implementation, thereby achieving hardware flexibility and efficiency. Evaluation results show that 34.08%∼55.41% operation reduction were achieved on six CNN models, while incurring a slight hardware overhead.**

***Index Terms*— Convolutional neural networks, acceleration algorithm, convolution decomposition, flexibility, hardware-efficient.**

## I. Introduction

CONVOLUTIONAL neural networks (CNNs) have been widely applied in various computer vision tasks, including image classification [16], object recognition [17], [19], [20] and semantic segmentation [18], [21], [22]. However, a deep CNN model typically has ten to hundreds of convolution layers with billions of multiply-and-accumulate (MAC) operations, which cause greatly challenges for the performance of acceleration hardware. In order to speed up the inference process, various emerging methods from different aspects have been used for CNN accelerator design, including sparsification in weights and activations [11], [27], [28], low-bit data precision [12], [29], convolutional acceleration algorithms [1], [2], [7], [10], and etc. The major computational complexity of typical CNN models lies in the computing-intensive convolution layers. Therefore, an efficient way to accelerate CNNs is to reduce the amount of multiplication operations by using convolutional acceleration algorithms, such as Winograd [2], FFT [10], and FFA [7]. Taking Winograd algorithm as an example, Fig.1 and Fig.2 show the pseudo code of conventional convolution and Winograd. In Fig.2, the variables $F$, $In$, and $Out$ represent the filter, input tile and output result of Winograd algorithm, respectively; the variables $G$, $B$ and $A$ are the transformation matrices for variables $F$, $In$, and $Out$, respectively. First, filter $F$ is transformed by matrices $G$ and $G^T$ (i.e, the transpose of $G$), and input tile $In$ is transformed by matrices $B$ and $B^T$ (i.e, the transpose of $B$); as a result, the variables $U$ and $V$ represent the transformed $In$ and $F$. Next, the elements of $U$ and $V$ execute the element-wise multiplication. Finally, the dot-product result is transformed by matrices $A$ and $A^T$ (i.e, the transpose of $A$), thereby $Out$ is obtained. Compared to conventional convolution, Winograd has fewer loops (i.e., eliminating two inner loops), thereby saving much multiplication operations.

```
for(row=0;row<H;row+=s)                                          //row_loop
    for(col=0;col<W;col+=s)                                      //col_loop
        for(ti=0;ti<M;++ti)                                      //ich_loop
            for(to=0;to<N;++to)                                  //och_loop
                for(i=0;i<r;++i)                                 //filter_loop
                    for(j=0;j<r;++j)
{output[row][col][to]+=filter[ti][to][i][j]*input[row+i][col+j][ti]}
```

Fig. 1. Pseudo code of conventional convolution.

These three convolutional acceleration algorithms (i.e., FFT, Winograd, and FFA) have been widely used in recent CNN accelerators [1], [7]–[10], [13], [26], [34]. Generally, Winograd often performs well for the convolution shapes of small kernel sizes; while FFT is more efficient for large kernel sizes; FFA doesn't perform as well as Winograd but having advantage of less pre-computations. However, three issues still exist in these acceleration algorithms when implementing

Manuscript received December 9, 2019; revised March 7, 2020; accepted March 30, 2020. Date of publication April 22, 2020; date of current version September 2, 2020. This work was supported in part by NNSF of China under Grant 61704136, in part by the China Postdoctoral Science Foundation under Grant 2018M631163, in part by the Shaanxi Postdoctoral Research Project Foundation under Grant 2017BSHEDZZ29, in part by the Fundamental Research Funds for the Central Universities under Grant Z201805200, in part by the Guangdong Province Key R&D projects under Grant 2019B010154002, and in part by the Key Research and Development Plan of Shaanxi Province under Grant 2019ZDLGY03-07-01. This article was recommended by Associate Editor J.-Y. Kim. *(Corresponding author: Chen Yang.)*

The authors are with the School of Microelectronics, Xi'an Jiaotong University, Xi'an 710049, China (e-mail: chyang00@xjtu.edu.cn; wangyizhou@stu.xjtu.edu.cn; xlwang@xjtu.edu.cn; gengli@xjtu.edu.cn).

Color versions of one or more of the figures in this article are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TCSI.2020.2985727

```
for(row=0;row<H;row+=m)                                    //row_loop
    for(col=0;col<W;col+=m)                                //col_loop
        for(ti=0;ti<M;++ti)                                //ich_loop
            for(to=0;to<N;++to)                            //och_loop
{                   load:tile_Z(row,col,ti);
                    load:filter[ti][to];
                    Winograd(tile_Z,filter,tile_Y);
                    output(row,col,to)+=tile_Y(row,col,to); }
    Winograd(In,F,Out){U=B^TInB,V=GFG^T;Out=A^T(U⊙V)A};
```

Fig. 2. Pseudo code of Winograd acceleration algorithm.

an efficient hardware accelerator. Firstly, these convolutional acceleration algorithms are hardly implemented on a uniform computation hardware, due to different transformation matrices and uneven size of input tiles. As a result, in order to accelerate different convolution shapes with $N$ types of kernel sizes, $N$ computational hardware units should be designed. Like [7], it designed 3 types of FFA units to supports 3/5/7 kernel sizes. When one of the three units is working, the other two units are left unused. Consequently, such implementation causes much redundancy if many different units are built to support multiple kinds of kernel sizes. Secondly, using larger parameter for convolutional acceleration algorithms can usually achieve higher yield of multiplication reduction but will incur larger precision loss on an 8 or 16 bits fixed point data representation, which are commonly used in CNN accelerators. Therefore, current hardware accelerators tend to choose small parameters when using these convolutional acceleration algorithms. For example, Winograd *F(4,2)* unit and Winograd *F(4,3)* unit was employed in work [8] and work [1], respectively, to avoid unacceptable data precision loss. As a result, they are hardly to support large kernel sizes. Thirdly, these three acceleration algorithms are typically designed aiming at 1-stride convolutions, which is the major restriction to the flexibility of hardware implementation. As a result, most of state-of-the-art works [1], [7], [8], [9] that using convolutional acceleration algorithms can only support 1-stride convolution shapes and hardly carry out large strides. To the best of our knowledge, there are little works that focus on extending these three convolutional acceleration algorithms for accelerating large-stride convolutions.

In this paper, we firstly proposed a Stride-based Convolution Decomposition Method (SCDM) to stretch the use range of Winograd for hardware-flexible implementation. In SCDM, different convolution shapes are decomposed and reformed to a uniform kernel size and stride, and then all the decomposed results can be fed into the same one computation unit, which is accelerated by Winograd algorithm. With the aid of SCDM, a uniform Winograd-based computation unit is employed to accelerate convolutions with any kernel size and any stride so that improving the area efficiency of hardware implementation. Next, an Area Efficiency Model (AEM) was put forward to screen out proper parameter for efficient implementation of the Winograd-based computation unit. Also, a Multiplication Consumption Model (MCM) is built to accurately evaluate the multiplication saving rate brings by the Winograd-based computation units. The selected computation unit makes a trade-off among hardware overhead, precision loss, performance enhancement and energy saving. At last, a Winograd-stretched and Hardware-efficient Design (WHD) was presented by fusing two efficient Winograd-based units, which enhances the flexibility of traditional convolutional acceleration algorithms and achieves an area-efficient computation architecture. The key ideas behind WHD are to employ SCDM for decomposing different convolution shapes into an identical pattern and utilize the complementary advantages of two computational infrastructures.

Overall, the main contributions of this paper are as follows.

- In SCDM, different convolution shapes are decomposed to an identical kernel size and stride, which can be fed into a uniform computation unit accelerated by Winograd. SCDM is beneficial for improving area-efficient of hardware and extending the use range of convolutional acceleration algorithm for various CNN shapes.
- AEM and MCM models are put forward for selecting the applicable parameters for the uniform computation unit to reduce precision loss and improve hardware sharing, as well as building an accurate performance evaluation of computation unit.
- WHD utilizes the complementary advantages of two fusing Winograd-based units. With the aid of SCDM, WHD improve the flexibility and area efficiency of computation architecture, still providing a better performance for a large spectrum of CNN models compared to previous Winograd/FFT/FFA-based works.

The rest of the paper is organized as follows. Section II summarizes related work and explains the motivation. Section III describes proposed SCDM and WHD. Section IV evaluates the WHD unit for different CNN shapes and models. Section 0 presents the conclusion.

## II. Related Work and Motivation

In this section, the recent works using convolutional acceleration algorithms are provided and discussed. Then the motivations of proposed SCDM method and WHD architecture are explained in detail

### A. Related Work

To improve performance, three convolutional acceleration algorithms, i.e., Winograd, FFT and FFA, have been widely used and implemented in recent CNN accelerators. In order to support various kernel sizes (e.g., 3, 5, and 7), a FFA-based accelerator [7] have to build three different types of FFA computation units. However, such simply stacking multiple computing units causes significant hardware overhead and power consumption in hardware implementation. Aiming at avoiding large data precision loss, current works [4], [9] and [8] tend to used Winograd *F(4,3)* or *F(4,2)* units, respectively. However, a single computation unit lacks flexibility and can only accelerate a certain type of convolution shapes. Besides, to achieve more multiplication reduction, some works [1], [10], and [35] employed large-parameter Winograd units (i.e., *F(6,3)* or *F(8,5))*. As a result, larger precision loss is inevitable, as well as more hardware resource involved. Recent work [34] deploys four types of Winograd units, including *F(8,3)*, *F(6,3)*, *F(5,3)* and *F(4,3)*, to improve the computation efficiency for $3 \times 3$ convolution shape, along with GEMM-based (general element-wise matrix multiplication) method for other kernel

TABLE I
THE COMPUTATION UNIT TYPE BUILT IN DIFFERENT ACCELERATORS

| Accelerator | Accelerate Algorithm | Computation Unit |
|---|---|---|
| 2017 FCCM [1] | Winograd | F(6,3) or F(8,5) |
| 2018 ICSICT [9] | Winograd | F(4,3) |
| 2018 TCAS-I [7] | FFA | F(3), F(5), F(7) |
| 2017 ICFPT [8] | Winograd | F(4,2) |
| 2019 TCAD [10] | FFT | N=8 |
| 2019 TVLSI [34] | Winograd | F(8,3), F(6,3), F(5,3), F(4,3) |

TABLE II
THE ELEMENTS IN WINOGRAD TRANSFORMATION MATRIX

| Tile size n | Kernel size r | Stride s | Max number | Min number |
|---|---|---|---|---|
| 4 | 3 | 1 | 1 | 1/2 |
| 5 | 3 | 1 | 4 | 1/6 |
| 6 | 3 | 1 | 8 | 1/24 |
| 7 | 5 | 1 | 16 | 1/120 |
| 8 | 5 | 1 | 49 | 1/720 |

sizes. Due to non-uniform Winograd units, work [34] has to compute transformed filter matrices offline. In contrast, some works [2], [13], [26], [36] considered to build FFT-based unit to flexibly accelerate various convolution shapes. Although FFT computation unit can be adapted to different kernel sizes and strides, FFT cannot bring satisfied multiplication reduction compared to Winograd especially when kernel size is small. Furthermore, some works [2], [33] deploy an integration of both FFT-based and Winograd-based units in their accelerators to better fit different kernel sizes. But, such combination of two different acceleration algorithms consumes large hardware resource on transformations computing and overlooks the kernel stride variation in rapidly evolving CNN models.

### *B. Motivation*

Using aforementioned convolutional acceleration algorithms to boost current CNN accelerators, there are some issues on flexibility and efficiency of hardware.

Firstly, when using Winograd or FFA, each computation unit with different parameters only matches a certain convolution shapes. As shown in TABLE I, these accelerators only involve few types of unit to save hardware resource and simplify the computation design, but resulting in weak flexibility. In our previous work [9], a Winograd-based CNN accelerator named WRA was designed. The processing element (PE) in WRA is based on Winograd *F(4,3)* unit, which brings about 55.6% multiplication reduction. However, like most other accelerators using convolutional acceleration algorithms, WRA only supports 1/3 kernel size and 1 stride. That is to say, multiple types of computation unit with different parameters needed to be built for extending the configurability of accelerator. In such way, large amount of hardware logic have to be employed for feature and filter transformations. Besides, when one types of computation unit is running, the others stop and don't work, which significantly decreases the area efficiency of accelerators.

Secondly, even if multiple computation units with different parameters are built to support large range of kernel size or stride, the large-parameter units will incur unacceptable data precision loss. Due to the essential quantization process of weights in hardware accelerators, the precision loss increases quickly when the parameters become larger. Taking Winograd as an example, the data range of transformation matrices are shown in TABLE II. For the example of $n = 4$, the transformation matrix is simple and consists of two non-zero elements (i.e., 1 or 1/2 ), which are easily to be quantized. In contrast, when the tile size is 8, the minimal number and the maximal number in transformation matrix are 1/720 and 49, respectively. Such large data range will inevitably bring large precision loss, especially in the widely used 8 or 16 bits fixed point data representation. To avoid this issue, most of previous accelerators choose computation units with small parameters. As a result, they are hardly to perform convolution shapes with large kernel sizes.

Thirdly, these convolutional acceleration algorithms are commonly used to speed up 1-stride convolution shapes, which severely restricts the flexibility of hardware implementation. Thus, there are some works [1], [10] intending to enlarge the flexibility of accelerators. However, the reduction rate of multiplication cannot always stay at the high level for different convolution shapes. For the example in work [1], the proposed accelerator used direct multiplication (i.e. conventional convolution) to support $11 \times 11$ kernel size and used zero padding to support $5 \times 5$ kernel size, but these solutions are inefficient due to lowering their multiplication reduction rate (e.g., 40.7% degradation in the case of Winograd *F(6,3)*). For another example of FFT-based accelerator [10], although different convolution shapes can be supported by using zero padding, it causes much extra multiplications ($4N^2$, where $N$ is the number of sampling in TABLE I). As a result, the FFT performs worse for small kernel size compared to Winograd. Although some optimization methods [14], [15] can be employed to cut down the extra multiplications to around $1.5N^2$, unfortunately, these methods are not hardware-efficient due to irregular distribution of symmetry elements in transformation matrices.

To solve the above three issues on hardware flexibility and area efficiency, this paper proposed a novel convolution decomposition method based on kernel stride, named SCDM. After decomposition, one uniform and area-efficient Winograd computational unit can accelerate convolutions with any kernel sizes and any strides, as well as maintaining a high level rate of multiplication reduction. With the help of SCDM, an algorithm-stretched and hardware-efficient prototype, named WHD, is presented. It fuses the complementary advantages of two Winograd-based units, thereby enhancing the flexibility of convolutional acceleration algorithms and achieving an area-efficient hardware architecture. The reason of using Winograd in SCDM and WHD is that Winograd is more effective for smaller kernel sizes compared to FFT. As reported in [2] and [4], FFT-based convolutions are inefficient for small kernel sizes if mismatch exists between kernel size and input size. Thus, the widespread use of 1/3/5 kernel sizes in emerging CNNs (e.g., ShuffleNet [25], MobileNet [23], [24], YOLO [30], and etc.) make Winograd be a preferable option. Besides, the typical multiplication reduction of Winograd (e.g., 57-69%) is better than that (e.g., 33-43%) of FFA [7].

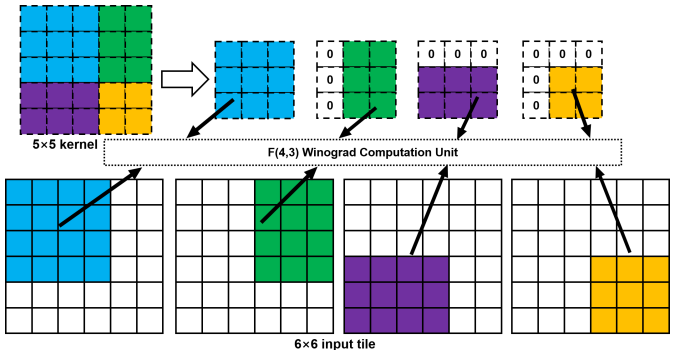



Fig. 3. Decomposition method for F(4,3) when s=1.

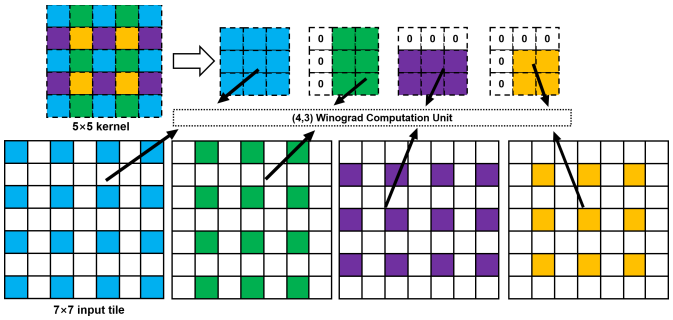



Fig. 4. Decomposition method for F(4,3) when s=2.

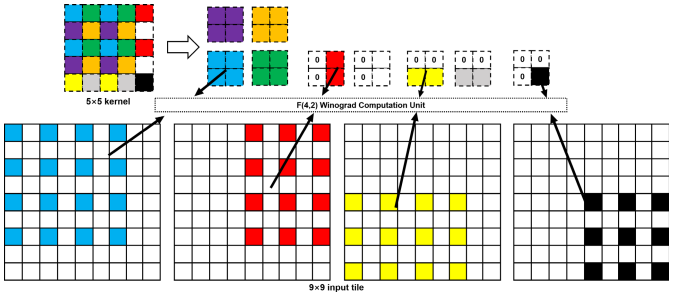



Fig. 5. Decomposition method for F(4,2) when s=2.

## III. Winograd-Stretched and Hardware-Efficient Design

In this section, the detailed design flow of the algorithm-stretched and hardware-efficient architecture, named WHD, is provided. Firstly, a stride-based convolution decomposition method, i.e., SCDM, is presented to improve the area efficiency and hardware flexibility, which aims at using a uniform computation unit based on a certain Winograd *F(m,n)* (where $m$ is the input tile size, and $n$ is the kernel size) to accelerate convolutions with any kernel sizes and any strides. Then, a universal area efficiency model and a multiplication consumption model for CNN accelerator is presented, along with the discussion on what parameters of Winograd *F(m,n)* units can achieve better trade-off between area efficiency and precision loss. Finally, to augment the acceleration yield of Winograd algorithm, a hardware-efficient architecture of unifying *F(4,3)* unit and *F(4,2)* unit together is designed to reach complementary advantages and substantial computation logic sharing.

### A. A Stride-Based Convolution Decomposition Method

Given there is only a certain Winograd *F(m,n)* computation unit, convolution with $n$ kernel size and 1 stride can be perfectly accelerated by this unit. However, when the shape of convolution is not matched with the *F(m,n)* computation unit, this unit cannot work. In SCDM, convolutions shapes with different kernel size $r$ and kernel stride $s$, will be decomposed and filled into several parts to fit in *F(m,n)* computation unit, and the results of these parts are summed to form an equivalent result. In this way, once a proper computation unit is chosen, we can use the unit to accelerate other shapes of convolution by SCDM. The novelty of the SCDM method is to use uniform hardware based on Winograd to accelerate any convolution shapes. In the following paragraphs, we use Winograd *F(4,3)* and *F(4,2)* as examples to describe the detailed working flow of SCDM under the circumstances of different convolution shapes.

Firstly, the basic case is that the kernel stride $s$ is equal to 1. Taking *F(4,3)* as an example, when kernel size $r$ is less than 3, the kernel should be filled into $3 \times 3$ with zeros. When $r$ is larger than 3, the kernel is decomposed and filled into several $3 \times 3$ kernels to fit for *F(4,3)* unit. The decomposed $3 \times 3$ kernels are composed of neighboring elements in the original kernel shown in Fig.3. In this example, when kernel size is 5, a $5 \times 5$ kernel is decomposed into one $3 \times 3$ block, two $2 \times 3$ blocks and one $2 \times 2$ block. The non-$3 \times 3$ blocks (i.e., $2 \times 3$ block and $2 \times 2$ block) are filled into $3 \times 3$ blocks with zeros padding. Accordingly, when the $5 \times 5$ kernel is sliding on the $6 \times 6$ input tile, each decomposed kernel block is mapped on the input tile, generating corresponding one $4 \times 4$ block, two $4 \times 3$ blocks and one $3 \times 3$ block, respectively. The decomposed kernel block and its corresponding region on the $6 \times 6$ input tile are labeled using the same color in Fig.3. To match the $4 \times 4$ input tile size required by *F(4,3)* unit, these $4 \times 3$ and $3 \times 3$ blocks in the $6 \times 6$ input tile are all filled into $4 \times 4$ blocks with zeros padding. Then, these four groups of input tile and kernel all fit into the *F(4,3)* computational unit. Finally, the four $2 \times 2$ outputs from four computational units are summed together.

Next, the situation that $s$ is equal to 2 is present, also using *F(4,3)* as an example. When kernel size $r$ is less than 3, same as $s = 1$, the kernel should be filled into $3 \times 3$ with zeros; When $r$ is larger than 3, the kernel is decomposed and filled into several $3 \times 3$ kernels. However, instead of gathering neighboring elements into a decomposed kernel as shown in the case of $s = 1$, the elements with 2 steps distance both in the vertical and horizontal direction are gathered. For the example in Fig.4, when kernel size is 5, a kernel is also decomposed into one $3 \times 3$ block, two $2 \times 3$ blocks and one $2 \times 2$ block, but in different location from the original kernel. Similarly, sliding the $5 \times 5$ kernel with 2 stride on the $7 \times 7$ input tile, the same color in the input tile is gathered to form the decomposed input tile blocks. Both the decomposed kernel and input tile are filled into $3 \times 3$ and $4 \times 4$ size with zeros padding if necessary. Taking *F(4,2)* unit in Fig.5 as another example, the difference from *F(4,3)* unit is that the kernel blocks from decomposition should be smaller than $2 \times 2$ size. As a result, nine decomposed kernels blocks are generated, some of which are filled into $2 \times 2$ kernel with zeros padding. Correspondingly, the $9 \times 9$ input tile is decomposed into nine blocks that are all filled into $4 \times 4$ input tiles.

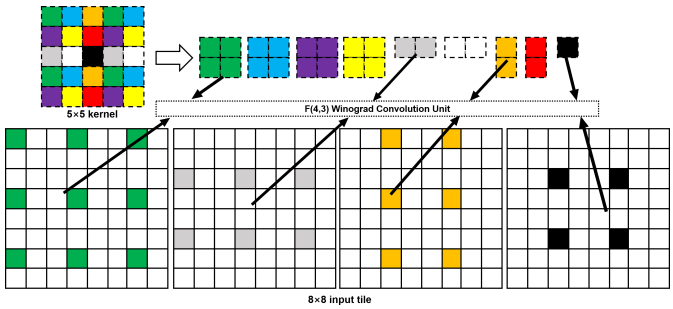



Fig. 6. Decomposition method for F(4,3) when s=3.

Naturally, aforementioned decomposition method can be easily generalized for broader cases where $s$ becomes larger. Fig.6 shows the situation where $s$ is equal to 3. In this example, the elements with 3 steps distance in both kernel and input tile are gathered. Thus, nine groups of small blocks are generated. In the same way, when *F(4,3)* unit is used, nine kernel blocks are filled into $3 \times 3$ size and nine input blocks are filled into $4 \times 4$ size using zeros padding. Total of nine Winograd computation units is required to finish this convolution shape ($r = 5$ and $s = 3$). In the circumstance that *F(4,2)* unit is used in the same example, the last five out of the nine blocks is filled into $2 \times 2$ size using zeros padding. The "jumping mechanism" used in gathering elements when $s > 1$ can generate the smallest-size kernel and input tile after decomposition, so that no redundant computation is incurred in the Winograd.

Generally, no matter what the values of $r$ and $s$ are, the kernel and input tile can be decomposed and filled into a uniform shape fitting for a certain *F(m,n)* unit. The kernel stride $s$ determines the decomposition method, in other words which elements is selected for gathering together. Both the kernel size $r$ and stride $s$ determines the number of blocks generated after decomposition. More importantly, such decomposition process is totally periodical and regular, which is convenient for data organization and fetching when hardware implementation. In this way, using the SCDM method can accelerate convolution shapes with any kernel size and stride in a certain Winograd *F(m,n)* computation unit. To sum up, the generalized working flow of SCDM is described step by step as follows. Also, the pseudo code is provided in TABLE III.

Step-i If $r$ is less than or equal to $n$, $r \times r$ kernel is filled into $n \times n$ size directly without decomposing, and so does input tile.

Step-ii If $r$ is larger than $n$, the kernel is decomposed into several $n1 \times n2$ ($n1, n2 < n$) blocks, each of which consisted of $s$ steps distance elements.

Step-iii The input tile is decomposed into several $m1 \times m2$ ($m1, m2 < m$) blocks, each of which consisted of $s$ steps distance elements.

Step-iv The decomposed kernel blocks and input blocks are filled into $n \times n$ and $m \times m$ matrices with zeros padding, respectively.

Step-v Each decomposed kernel and its associative input forms a group, which is processed in a Winograd convolution unit.

TABLE III

THE PSEUDO CODE OF SCDM METHOD

Stride-based Convolution Decomposition Method (SCDM)

**If** $r \leqslant n$ /*The situation when $r$ is less than or equal to $n$*/
  $N=1$ /*Set the number of decomposed block to 1*/
  $F_0$= zeros(*1:n, 1:n*) /*Initiate a kernel block as an $n \times n$ zero matrix*/
  $F_0(n\text{-}r+1:n, n\text{-}r+1:n)=K(1:r, 1:r)$ /*Put original $r \times r$ kernel $K$ into $F_0$*/
  $In_0$= zeros(*1:m, 1:m*) /*Initiate a input block as an $m \times m$ zero matrix*/
  $In_0(1:m, 1:m)=IN(1:m, 1:m)$ /*Put original $m \times m$ input tile $IN$ into $In_0$*/
**Else** /*The situation when $r$ is larger than $n$*/
  **If** $s \times n > r > s$
    $N=s^2$ /*Calculate the number of decomposed blocks*/
    $N\_sqrt=s$
  **Else**
    $N=\left\lceil \frac{r}{n} \right\rceil^2$ /*Calculate the number of decomposed blocks*/
    $N\_sqrt=\lceil r/n \rceil$
  **End**
  **For**($i=0; i \leqslant N\text{-}1; i$++) /*Generate $N$ decomposed kernel blocks $DF_i$*/
    **For**($j=1; j \leqslant r; j$++) /*Get elements for each kernel block*/
      *if* $(i\%N\_sqrt < s)$ $n1=i\%N\_sqrt+1+(j\text{-}1) \times s$
      *else* $n1=(i\%N\_sqrt) \times n+1+(j\text{-}1) \times s$
      **If** $n1 \geqslant n$ /*Keep kernel block's width less than $n$*/
        $n1_i=n1$
        **Break**
      **Else**
        **For**($k=1; k \leqslant r; k$++)
          *if* $(\lfloor i/N\_sqrt \rfloor < s)$ $n2=\lfloor i/N\_sqrt \rfloor+1+(k\text{-}1) \times s$
          *else* $n2=\lfloor i/N\_sqrt \rfloor \times n+1+(k\text{-}1) \times s$
          **If** $n2 \geqslant n$ /*Keep kernel block's height less than $n$*/
            $n2_i=n2$
            **Break**
          **Else**
            $DF_i(j, k)=K(n1, n2)$ /*Get elements of $DF_i$*/
          **End**
        **End**
      **End**
    **End**
  **End**
  **For**($i=0; i \leqslant N\text{-}1; i$++) /*Generate $N$ decomposed input blocks $DIn_i$*/
    **For**($j=1; j \leqslant r+(m\text{-}n) \times s; j$++) /*Get elements for each input block*/
      *if* $(i\%N\_sqrt < s)$ $m1=i\%N\_sqrt+1+(j\text{-}1) \times s$
      *else* $m1=(i\%N\_sqrt) \times m+1+(j\text{-}1) \times s$
      **If** $m1 \geqslant m$ /*Keep input block's width less than $m$*/
        $m1_i=m1$
        **Break**
      **Else**
        **For**($k=1; k \leqslant r+(m\text{-}n) \times s; k$++)
          *if* $(\lfloor i/N\_sqrt \rfloor < s)$ $m2=\lfloor i/N\_sqrt \rfloor+1+(k\text{-}1) \times s$
          *else* $m2=\lfloor i/N\_sqrt \rfloor \times m+1+(k\text{-}1) \times s$
          **If** $m2 \geqslant m$ /*Keep input block's height less than $m$*/
            $m2_i=m2$
            **Break**
          **Else**
            $DIn_i(j, k)=IN(m1, m2)$ /*Get elements of $DIn_i$*/
          **End**
        **End**
      **End**
    **End**
  **End**
  **For**($i=0; i \leqslant N\text{-}1; i$++) /*Fill kernel blocks and input blocks into $n \times n$ and $m \times m$ matrices with zeros padding, respectively*/
    $F_i$=zeros(*1:n, 1:n*)
    $F_i(n\text{-}n1_i+1:n, n\text{-}n2_i+1:n)=DF_i(1:n1_i, 1:n2_i)$
    $In_i$=zeros(*1:m, 1:m*)
    $In_i(m\text{-}m1_i+1:m, m\text{-}m2_i+1:m)=DIn_i(1:m1_i, 1:m2_i)$
  **End**
**End**
**For**($i=0; i \leqslant N\text{-}1; i$++) /*Process each combination of decomposed kernel and input in Winograd *F(m,n)* convolution unit, and accumulate output*/
  $Out_i$=Winograd($In_i$, $F_i$)
  $Result=Result+Out_i$
**End**

Step-vi Accumulating Winograd convolutional result of each decomposed kernel and its associative input.

The SCDM method can be performed in two approaches, i.e., off-line and on-line. The off-line approach is to process the decomposition of input tile and kernel off the chip. That is to say, both input tile and kernel can be decomposed into smaller blocks to fit an identical Winograd *F(m,n)* unit before they are saved in the on-chip buffer. In this approach, no complex control logic for decomposition is required, and just the decomposed input tile and kernel are stored in the on-chip buffer. However, some overlapping regions may exist between decomposed input tiles. Taking the $6 \times 6$ input tile in Fig.3 as an example, the $4 \times 4$ blue block and $4 \times 3$ green block share one column of overlapping region. Inevitably, such overlapping region cause extra data accesses from off-chip memory, as well as storage overhead compared to conventional GEMM method. The on-line approach is to deploy the decomposition of input tile and kernel on the chip. In this approach, the undecomposed input tile and kernel are store on the chip. As for a certain convolution shape, the data access pattern for decomposition is regular according to a certain stride. In this situation, the control logic for data access is not complex. However, the control logic will become complex as the usable range of SCDM method increases (i.e., supporting more kernel sizes & strides). This is because that the control logic of decomposition performs access address generation for diverse combinations of kernel sizes and strides. As a contrast, well-known CNN accelerators, such as Eyeriss [41] and Thinker [40], are typically reconfigurable architecture with conventional GEMM method. To support more convolution shapes, such accelerators have to employ complex interconnection between PEs. In contrast to these conventional CNN accelerators, using SCDM method can greatly simplify the interconnection between PEs due to an identical kernel size and stride after decomposition. Using on-line approach, the SCDM method requires no extra storage overhead for undecomposed input tile and kernel. Although input/filter/output transformations are involved in Winograd algorithm, no extra storage is needed for transformation matrices because these transformations are performed by hardwired logic. Besides, the overlapping regions existed between decomposed input tiles do not induce extra data accesses from off-chip memory because the data accesses for overlapping regions hit in the on-chip buffers of input tile and kernel.

### B. Exploring the Efficient F(m,n) Unit

SCDM can extend one *F(m,n)* unit to support wide range of convolution shapes. However, a certain *F(m,n)* unit can't always achieve the most multiplication reductions for any convolution shapes. To find out efficient Winograd-based computation unit for hardware implementation, two analytical models are built in this subsection. One model is an area efficiency model (AEM) of Winograd *F(m,n)* computation unit, which is used for a comprehensive evaluation on performance, area overhead, and precision. The other model is a multiplication consumption model (MCM) for SCDM and conventional convolution, which is used to accurately evaluate the multiplication saving rates of different *F(m,n)* computation units.

*1) Area Efficiency Model (AEM):* As described in equation (1), the area efficiency (*AE*) of a CNN accelerator is a quotient of performance (*Perf*) and hardware overhead (*Area*). The main computational complexity of CNN model exists in convolution layers. Thus, the performance can be approximate as a quotient of MAC operation number (*OPs*) and processing time, which comprises two divisions, i.e., memory access delay ($T_{mem}$) and computing time ($T_{comp}$). As for a Winograd-based accelerator, the area cost consists of dot multiplication logic ($A_{dot}$) and Winograd transformation logic ($A_{trans}$).

$$AE = \frac{Perf}{Area} = \frac{\frac{OPs}{T_{mem}+T_{comp}}}{A_{dot} + A_{tran}} \tag{1}$$

The computing time can be further detailed depicted by equation (2) in the context of Winograd-based accelerator. Here, $P_{in}$ is the parallelization degree of the input channels, and $P_f$ is the parallelization degree of the output channels. These two computational parallelization are both scalable as the number of Winograd computation units. $C_1$ and $C_2$ represent the number of input channel and output channel, respectively. $N_{slide}$ is the iteration number of sliding steps when the input tile of *F(m,n)* unit completes one row traversal on input feature. $N_{slide}$ is calculated by input feature size $W$ and kernel size $r$, as well as parameters in *F(m,n)* unit. There are two typical padding modes in Tensorflow, i.e., 'same' and 'valid'. If the padding mode is set as 'same', full input size is considered and the output size will be $W$ (if stride is 1). If the padding mode is set as 'valid', partial input size is included. Here, the padding mode used in equation (2) is 'valid', so that the output size [*W-(r-1)*] is different from input size $W$.

$$T_{comp} = \frac{C_1 \times C_2}{P_{in} \times P_f} \times {N_{slide}}^2 = \frac{C_1 \times C_2}{P_{in} \times P_f} \times \left[\frac{[W-(r-1)]}{(m-r+1)}\right]^2 \tag{2}$$

As depicted by equation (3), the area of *F(m,n)* unit can be further represented with the sum of multiplier area and adders area, which are both the functions of $m$ and $n$. Here, $U_{mult}$ is the logic area of one basic multiplier unit, and $U_{add}$ is the logic area of one basic adder unit.

$$Area = m^2 \times U_{mult} + [2m^2(m-1) + 2mn(m+n-2)] \times U_{add} \tag{3}$$

According to the AEM model built in equations (1)-(3), it is clear that the area efficiency and computational precision of Winograd computation unit substantially affected by the parameters of *F(m,n)*. It is necessary to find proper parameters $n$ and $m$ for reducing computing time meanwhile don't involving too much pre-computation logic and data precision loss. In this work, *F(4,3)* and *F(4,2)* units are selected as candidates based on the following three considerations.

The first consideration is improving area efficiency. $T_{mem}$ is the memory access delay for reading a input feature and can be considered as a constant for different *F(m,n)* units. Thus, according to equations (1) and (2), performance is highly relevant to the iteration number of sliding windows (i.e., $N_{slide}$). $N_{slide}$ is reduced by the Winograd acceleration algorithm, which achieves a reduction of approximate *(m-r+1)* times compared to conventional convolution. Thus, the larger

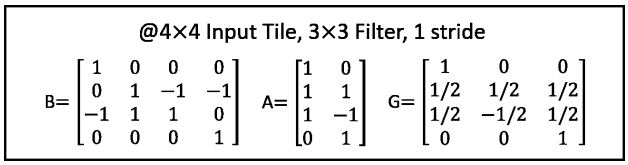



Fig. 7. The transformation matrices of Winograd F(4,3).

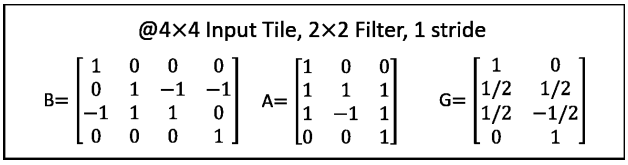



Fig. 8. The transformation matrices of Winograd F(4,2).

benefit of Winograd can be realized by using a larger input tile to obtain a smaller $N_{slide}$. However, the area of *F(m,n)* units expands as a squared or cubic function of $m$ and $n$. As a result, the area efficiency rapidly degrades as the enlargement of *F(m,n)* unit. Hence, small parameters of *F(m,n)* unit contribute to improve area efficiency. In contrast to *F(4,3)* and *F(4,2)* units, if much smaller parameter is used (e.g., *F(3,2)*), Winograd algorithm doesn't provide sufficient multiplication reduction.

Next, precision loss can be improved by smaller parameters of *F(m,n)* units. Computational precision largely depends on the quantization accuracy of fixed point data representation used in hardware implementation. Large parameters $n$ and $m$ will make the constant transformation matrices in Winograd too complex to be implemented on the hardware, which brings non-negligible quantization loss when using lower fixed point data representation. To reduce the complexity of pre-computation, the transformation matrices should be simple and sparse. Among all parameters, only when $n \leq 3$ and $m \leq 4$ can this condition be satisfied. As shown in Fig.7 and Fig.8, the transformation matrices of *F(4,3)* and *F(4,2)* units are both consisted of simple elements (i.e., 0, 1 or 1/2 ). Thereby the quantization loss can be avoided when adopting widely used 8 bits fixed point data representation. Also, the simple elements in *F(4,3)* and *F(4,2)* transformation matrices can simplify the multiplication of pre-computation into shift operation, which is conductive to further reduce area overhead.

As for the third consideration, the power (named as $P_{unit}$) of Winograd *F(m,n)* computation unit typically contributes the most part of the total power consumption and is highly relevant with the input tile size and kernel size of computation unit (i.e., parameters $m$ and $n$ in *F(m,n)*). On the same hardware platform (e.g., using a same FPGA board), $P_{unit}$ always increases as $n$ or $m$ increases. This is because that the larger parameters $n$ and $m$ involve much more pre-computation logic in hardware. Thus, small parameters used in *F(4,3)* and *F(4,2)* units are beneficial to reduce the computation units' power.

*2) Multiplication Consumption Model (MCM):* With the aid of SCDM, using Winograd *F(4,3)* or *F(4,2)* unit can support convolution shapes with any kernel size $r$ and stride $s$. In order to accurately calculate the multiplication saving rate of these two Winograd *F(m,n)* computation units, a generalized MCM is built for different convolution shapes. For general case, the numbers of multiplication operation that conventional convolution method and SCDM employed are presented by the equations (4) and (5) respectively, under the circumstances that these two methods generate the same size of output data.

TABLE IV

COMPARISON OF F(4,3) AND F(4,2) WHEN S=1

| | *F*(4,3) | | | *F*(4,2) | | |
|---|---|---|---|---|---|---|
| Kernel Size | Conventional[a] | SCDM | Saving Rate | Conventional[a] | SCDM | Saving Rate |
| 1 | 4 | 16 | ---[c] | 9 | 16 | --- |
| 2 | 16 | 16 | 0.0% | 36 | 16 | 55.6% |
| 3 | 36 | 16 | 55.6%[b] | 81 | 64 | 21.0% |
| 4 | 64 | 64 | 0.0% | 144 | 64 | 55.6% |
| 5 | 100 | 64 | 36.0% | 225 | 144 | 36.0% |
| 6 | 144 | 64 | 55.6% | 324 | 144 | 55.6% |
| 7 | 196 | 144 | 26.5% | 441 | 256 | 42.0% |
| 8 | 256 | 144 | 43.8% | 576 | 256 | 55.6% |
| 9 | 324 | 144 | 55.6% | 729 | 400 | 45.1% |
| 10 | 400 | 256 | 36.0% | 900 | 400 | 55.6% |
| 11 | 484 | 256 | 47.1% | 1089 | 576 | 47.1% |
| 12 | 576 | 256 | 55.6% | 1296 | 576 | 55.6% |

a. One computation of *F(4,3)* unit generate 4 point, and *F(4,2)* unit generate 9 point. Thus, the multiplication number based on Conventional method is different.
b. The highlight number represents the better performance between *F(4,3) unit and F(4,2)* unit.
c. "---" means conventional method performs better than SCDM.

TABLE V

COMPARISON OF F(4,3) AND F(4,2) WHEN S=2

| | *F*(4,3) | | | *F*(4,2) | | |
|---|---|---|---|---|---|---|
| Kernel Size | Conventional | SCDM | Saving Rate | Conventional | SCDM | Saving Rate |
| 3 | 36 | 64 | --- | 81 | 64 | 21% |
| 4 | 64 | 64 | 0% | 144 | 64 | 55.6% |
| 5 | 100 | 64 | 36% | 225 | 144 | 36.0% |
| 6 | 144 | 64 | 55.6% | 324 | 144 | 55.6% |
| 7 | 196 | 144 | 26.5% | 441 | 256 | 42.0% |
| 8 | 256 | 144 | 43.8% | 576 | 256 | 55.6% |
| 9 | 324 | 144 | 55.6% | 729 | 400 | 45.1% |
| 10 | 400 | 256 | 36.0% | 900 | 400 | 55.6% |
| 11 | 484 | 256 | 47.1% | 1089 | 576 | 47.1% |
| 12 | 576 | 256 | 55.6% | 1296 | 576 | 55.6% |

$$NumMult_{Convention} = r^2 \times (m-n+1)^2 \tag{4}$$

$$NumMult_{SCDM} = \begin{cases} s^2 \times m^2, s \times n > r \\ \lceil \frac{r}{n} \rceil^2 \times m^2, r \geq s \times n \end{cases} \tag{5}$$

The multiplication number of conventional convolution ($NumMult_{Convention}$) is the production of one kernel's multiplication (i.e., $r^2$) and the iteration number (i.e., $(m-n+1)^2$). In contrast, the multiplication operation involved in SCDM ($NumMult_{SCDM}$) is a piecewise function. When $r<s\times n$, $s^2$ kernels should be filled into $n\times n$ size, then they slide in $m\times m$ input tile and consume $s^2\times m^2$ multiplications. When $r \geq s\times n$, the kernel should be decomposed and filled into $\lceil \frac{r}{n} \rceil^2$ ($\lceil$ means ceiling) kernel with $n\times n$ size, as a result, each kernel consumes $m^2$ multiplications.

Based on equations (4) and (5), the multiplication saving rates of *F(4,3)* and *F(4,2)* units using SCDM compared to conventional convolution are calculated and listed in TABLE IV and TABLE V, respectively. TABLE IV shows the convolution shapes with $s = 1$, while TABLE V shows the situations when $s = 2$. The situation where $r < s$ is ignored because the information will be lost in the convolution and such convolution shapes never appear in current CNN models. The "saving rate" used in this paper means the rate of multiplication reduction when using SCDM method, compared to conventional GEMM method. The saving rate is calculated under the circumstance of these two methods generate the same size of output data, which ensures a fair comparison. Taking a convolution shape with $r = 3$ and $s = 1$ as an example, using SCDM method

TABLE VI
COMPARISONS BETWEEN F(4,3)/F(4,2) UNITS AND CONVENTIONAL METHOD

| Winograd unit | $F(4,3)$ | $F(4,2)$ |
|---|---|---|
| The condition SCDM performs **worse** than conventional method | $2s \geq r$ | $\frac{4}{3}s \geq r$ |
| The condition SCDM performs **better** than conventional method | $r > 2s$ | $r > \frac{4}{3}s$ |

with Winograd *F(4,3)* unit consumes 16 multiplications on the basis of equation (5) and produces $2 \times 2$ output data. To generate the same $2 \times 2$ output data size, the conventional GEMM method needs 36 multiplications according to equation (4). Thus, the saving rate in this example is calculated as (36-16)/36=55.6%. In these two tables, the better performance of these two competitive *F(4,3)* and *F(4,2)* units is highlighted as red color for each condition. Comparison shows that *F(4,2)* unit performs better than *F(4,3)* unit in most cases, which is due to smaller $N_{slide}$ used in *F(4,2)* unit. However, *F(4,3)* unit fits better in some widely used convolution shapes (e.g., $r = 3$ and $s = 1$), thereby performing much better than *F(4,2)* unit in these cases.

By observing the results listed in TABLE IV and TABLE V, the multiplication reduction of SCDM method consistently outperforms that of the conventional GEMM-based method, when kernel size $r$ exceeds a threshold under a given kernel stride $s$. The threshold is computed by equalize the equations (4) and (5). For *F(4,3)* unit, the threshold of $r$ is $2s$; and the threshold of $r$ for *F(4,2)* unit is $4s/3$. As a result, the benefits comparison between SCDM and conventional convolution is derived and listed in TABLE VI.

To generalize the comparison of saving rate between *F(4,3)* and *F(4,2)*, we conducted experiments on the combinations of $r$ and $s$ both ranging from 1 to more than 100, and counted the saving rate according to equations (4) and (5). Statistics on experimental results shows that *F(4,2)* unit is not inferior to *F(4,3)* unit when $3s > r > 4s/3$, and *F(4,3)* unit performs better than *F(4,2)* unit when $r \geq 3s$ and $r = 6k + 3$ ($k$ is natural number), besides, *F(4,2)* unit outperforms *F(4,3)* unit when $r \geq 3s$ and $r \neq 6k + 3$. In the example shown in Fig.9 (stride is set at 3), the serration fluctuation on the saving rate curve is consistent with the aforementioned statistical law. The saving rate on *F(4,3)* unit is superior to that of *F(4,2)* unit at the points of $r = 9$ and $r = 15$, which are the situations of $k = 1$ and $k = 2$.

### C. Stretch Winograd for Hardware-Efficient Implementation

According to AEM model, two Winograd units (i.e., *F(4,3)* and *F(4,2)*) are selected as candidates for implementation, and their performance variation is evaluated on MCM model for different convolution shapes. The key idea behind the proposed Winograd-stretched and hardware-efficient architecture (WHD) is to fuse *F(4,3)* unit and *F(4,2)* unit together so that they can complement each other's performance advantages, meanwhile their similar computation flow result in a uniform architecture with advantages of full logic sharing and high hardware reuse.

*1) Highly Reusable Hardware Architecture:* Fig.10 shows the overview architecture of WHD, which is a fusing

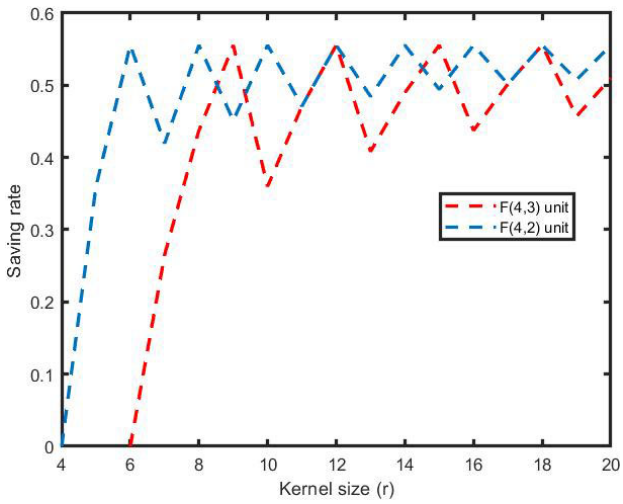



Fig. 9. The saving rate comparison between F(4,3) and F(4,2) unit for different convolution shapes with s=3.

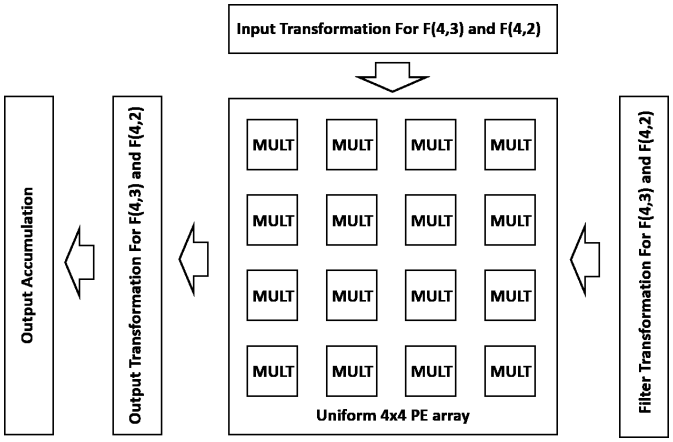



Fig. 10. The overview architecture of WHD fusing unit.

computation unit of Winograd *F(4,3)* and *F(4,2)*. As shown in Fig.7 and Fig.8, the transformation matrices of *F(4,3)* unit and *F(4,2)* unit are very similar, even the input transformation matrix $B$ of these two units is same. The other two transformation matrices $G$ and $A$ can have many same operators. Thus, the modules of input transformation, filter transformation and output transformation can accomplish the pre-computation of both *F(4,3)* unit and *F(4,2)* unit by highly sharing transformation logic, thereby involving little hardware overhead. The detailed structures of three transformation modules are demonstrated in Fig.11, Fig.12 and Fig.13, respectively. After the transformation of input and filter, the dot-product calculation of *F(4,3)* unit and *F(4,2)* unit are both on the basis of $4 \times 4$ shape. Thus, *F(4,3)* unit and *F(4,2)* unit can fully share a uniform $4 \times 4$ PE array to create the dot-product in parallel, where each PE is a multiplier. The results from the PE array are firstly transformed to obtain output result of Winograd algorithm for each combination of decomposed kernel and its associative input. Then, the accumulation module generates the final output of WHD, which is the element-wise accumulation of Winograd convolutional results for each combination of decomposed kernel and its associative input. The shape of final output is a $3 \times 3$ matrix for Winograd *F(4,2)*. As for Winograd *F(4,3)*, the valid final output is the top-left $2 \times 2$

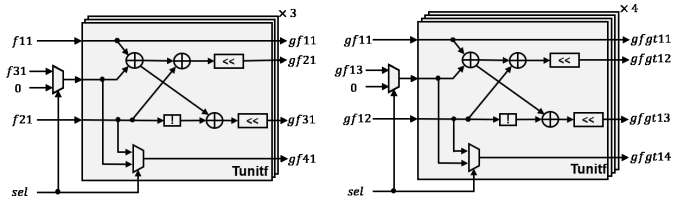



Fig. 11. The filter transformation module for both F(4,3) and F(4,2).

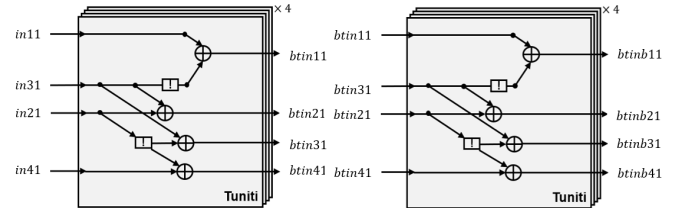



Fig. 12. The input transformation module for both F(4,3) and F(4,2).

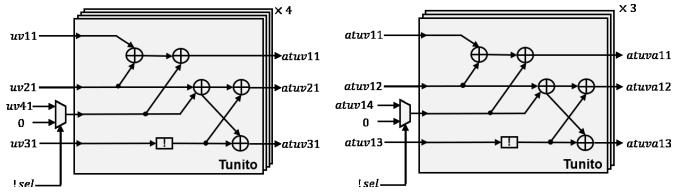



Fig. 13. The output transformation module for both F(4,3) and F(4,2).

block in the $3 \times 3$ matrix. To further improve the computation parallelism, multiple WHD element can be combined to form a scalable architecture for concurrent processing of multiple convolution channels.

Fig.11 shows the structure of filter transformation module, which employs seven *Tunitf* components to accomplish the computation of filter transformation in Winograd algorithm (i.e., $V{=}GFG^T$ in Fig.2). The input of filter transformation module are the elements of filter $F$ (e.g., *f11*, *f21*, and *f31*). The computational procedure of filter transformation module consists of two steps. First, three *Tunitf* components are employed to compute the multiplication of matrices $G$ and $F$ (i.e., $GF$), whose elements are the intermediate results of filter transformation module (e.g. *gf11, gf21, gf31, gf41, gf13,* and *gf12*). Next, four *Tunitf* components are deployed to accomplish the transformation of filter $F$ (i.e., $GFG^T$). The output of filter transformation module are the elements of transformed filter (e.g., *gfgt11, gfgt12, gfgt13,* and *gfgt14*), which are forward to the $4 \times 4$ PE array in the WHD architecture. Each *Tunitf* component employs three ADDs, two SHIFTs, one INV, and two MUX2s. The *sel* signal is used as the switch between *F(4,3)* and *F(4,2)* transformations, which is set at 1 when the *Tunitf* component is used for *F(4,3)*, and 0 for *F(4,2)*. Given a certain convolution shape, the *sel* signal is determined by aiming at choosing the best-performing one from *F(4,3)* and *F(4,2)* units. Specifically, the *sel* signal is set at 1 when $r \geq 3s$ and $r = 6k + 3$; else, the *sel* signal is set at 0 in the situations of $3s{>}r{>}4s/3$ or $r \geq 3s$ and $r \neq 6k + 3$.

Fig.12 shows the structure of input transformation module, which employs eight *Tuniti* components and is dedicated to compute input transformation in Winograd algorithm (i.e., $U{=}B^T InB$ in Fig.2). The input of this module are the elements of input tile *In* (e.g., *in11, in21, in31,* and *in41*). The computational procedure of input transformation module consists of two steps. First, four *Tuniti* components are employed to compute the multiplication of matrices $B^T$ and *In* (i.e., $B^T In$), where $B^T$ is the transpose of matrix $B$, and to generate the intermediate results of input transformation module (e.g. *btin11, btin21, btin31,* and *btin41*). Next, four *Tuniti* components are deployed to accomplish the transformation of input tile *In* (i.e., $B^T InB$). The output of input transformation module are the elements of transformed input tile (e.g., *btinb11, btinb21, btinb31,* and *btinb41*), which are forward to the $4 \times 4$ PE array in the WHD architecture. Each *Tuniti* component utilizes two INVs and four ADDs. Due to the input transformation matrix $B$ of *F(4,3)* and *F(4,2)* units is identical, both *F(4,3)* and *F(4,2)* use the same input transformation module, thereby no *sel* signal is needed in this module.

Fig.13 shows the structure of output transformation module which employs seven *Tunito* components and is used to process output transformation in Winograd algorithm (i.e., $Out{=}A^T(U \odot V)A$ in Fig.2). The input of this module is the dot-product of matrices $U$ and $V$ (e.g., *uv11, uv21, uv31,* and *uv41*), which comes from the $4{\times}4$ PE array in the WHD architecture. The computational procedure of output transformation module consists of two steps. First, four *Tunito* components are employed to compute the multiplication of matrices $A^T$ and PE array's results (i.e., $A^T(U \odot V)$), where $A^T$ is the transpose of matrix $A$, and to obtain the intermediate results of output transformation module (e.g. *atuv11, atuv21, atuv31, atuv12, atuv13,* and *atuv14*). Next, three *Tunito* components are deployed to accomplish the transformation of output result (i.e., $A^T(U \odot V)A$). The output of output transformation module (e.g., *atuva11, atuva12,* and *atuva13*) forms a $3 \times 3$ matrix, which is the result of Winograd *F(4,2)* convolution. As for Winograd *F(4,3)* convolution, only the top-left $2 \times 2$ block in the $3 \times 3$ matrix is valid. Each *Tunito* component consists of one INV, five ADDs and one MUX2. The *sel* signal is set at 1 when the unit is used for *F(4,3)*, and 0 for *F(4,2)*.

*2) Complementary Advantages on Performance:* Evaluation results on the MCM model demonstrate that Winograd *F(4,2)* performs better than Winograd *F(4,3)* in most cases, especially for situations of small kernel sizes. While Winograd *F(4,3)* substantially outperforms Winograd *F(4,2)* in some widely used convolution shapes, such as $r = 3$ and $s = 1$, thereby becoming an indispensable supplement. Therefore, the fusing computation structure of WHD can achieve yields from both *F(4,3)* unit and *F(4,2)* unit.

Fig.14 shows the detailed comparison of multiplications involved in *F(4,3)* unit, *F(4,2)* unit and fusing unit for different $r$ and $s$ parameters combinations. For all comparisons, the conventional GEMM-based convolution was used as the baseline, whose multiplication was normalized to 1. The result shows that the saving rates of fusing unit, *F(4,3)* unit and *F(4,2)* unit all decreases as stride increases, but *F(4,3)* unit degrades more quickly. *F(4,2)* unit performs almost as well as the fusing unit. This is because the decomposed size of *F(4,2)* unit is smaller than that of *F(4,3)* unit, so that less zero is padded, thereby less redundant multiplication involved. However, *F(4,3)* unit performs great when $r \geq 3s$ and $r = 6k + 3$, especially for $r = 3$ and $s = 1$, which is most widely used in the CNN model, such as VGG16 and AlexNet. In this situation, *F(4,3)* unit can achieve 55.6% multipliers reduction, much better than *F(4,2)* unit's 21% reduction. In Fig.14,

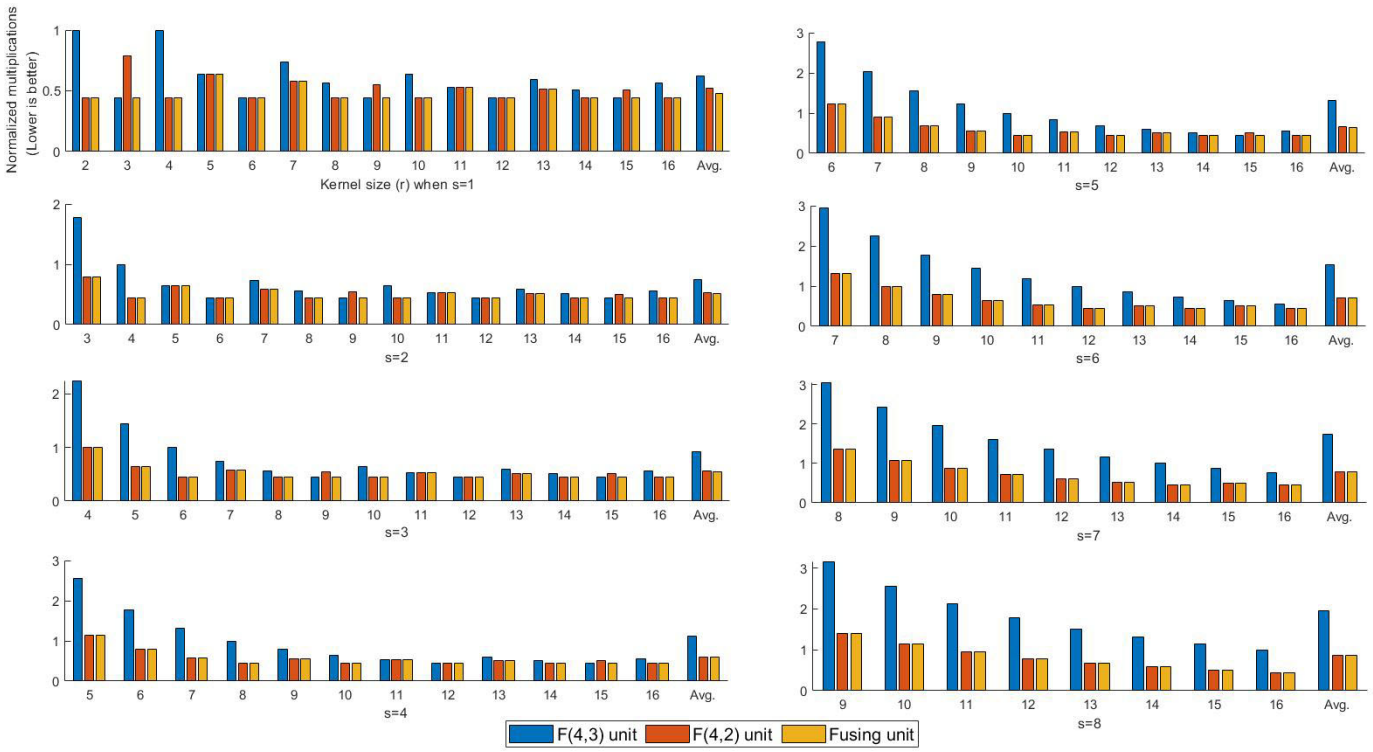



Fig. 14. Comparison of multiplications used in F(4,3) unit, F(4,2) unit and fusing unit for different convolution shapes (normalized to conventional GEMM-based convolution).

the bars labeled "Avg." represent the geometric mean of the individual saving rate. Geometric mean can avoid unduly influence on average performance caused by certain extreme high of individual performance, thereby can better reflect the stability of average performance gain. With stride increasing, the average multiplication saving rate of *F(4,3)* unit decreases quickly and performs worse than conventional GEMM-based convolution, but *F(4,2)* unit still provides more than 20% average saving rate if compared to GEMM-based convolution. Therefore, the fusing unit gets benefits from *F(4,2)* unit under the circumstance of large kernel size.

In Fig.14, convolution shapes under evaluation is $s + 1 \leq r \leq 16$ as stride $s$ scans from 1 to 8, which cover most of common convolution shapes in current CNN models. The goal of such evaluation settings is to assess the high flexibility and wide effectiveness of proposed SCDM method and WHD fusing unit. The larger kernel size and stride can increase the receptive field in feature extraction of convolution, as well as cutting down the computation of network. To the best of our knowledge, there are a few works using large kernel size or stride in some specific cases, such as semantic segmentation [37], encoding and decoding [38], and deconvolution [39]. However, in practice, the convolution shapes with 12∼16 size and 3∼8 stride are impractical since they rarely appeared in current CNN models, even most recent emerging CNN models have shown that kernel sizes are less than or equal to 5. Therefore, we count the average multiplication reduction of WHD fusing unit compared to *F(4,3)* and *F(4,2)*, when filter shapes under evaluation is $s + 1 \leq r \leq 11$ as stride $s$ scans from 1 to 2. Comparison results show that fussing unit achieves 18.7% and 9.0% improvements on multiplication reduction, if compared to *F(4,3)* and *F(4,2)*, respectively. Although only average 9.0% improvements compared to *F(4,2)* is achieved, for the most widely used convolution shape (i.e., $r = 3$ and $s = 1$), the WHD fusing unit can substantially outperform *F(4,2)* and save 43.8% multiplications compared with *F(4,2)*.

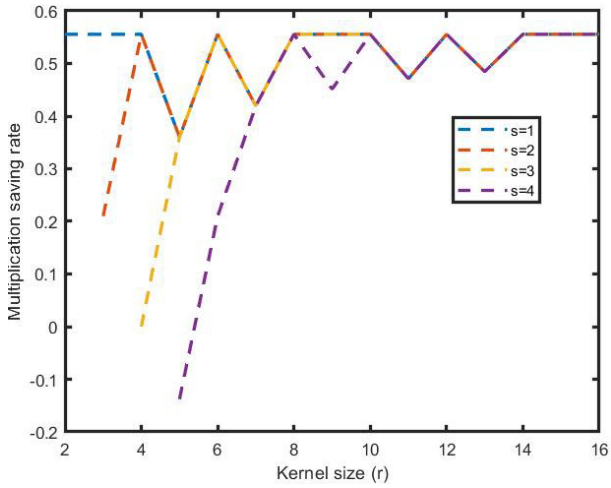



Fig. 15. The performance of WHD fusing unit for different convolution shapes.

## IV. Evaluation and Comparison

In this section, the performance of WHD fusing unit is firstly evaluated on different convolution shapes and CNN models. Next, comparisons with other acceleration methods from the views of multiplication saving, transformation complexity, flexibility and hardware uniformity are provided. At last, the hardware cost of WHD fusing unit is discussed and compared with other architectures.

TABLE VII
THE PERFORMANCE OF WHD FUSING UNIT FOR DIFFERENT CNN MODELS

| CNN models | Conventional OPs (MOPs) | WHD Ops (MOPs) | Saving Rate |
|---|---|---|---|
| AlexNet | 2219.6 | 1212.2 | 45.39% |
| VGG16 | 30800.2 | 13734.7 | 55.41% |
| YOLOV1 | 40358.7 | 19713.7 | 51.15% |
| DarkNet19 | 5582.0 | 2705.6 | 51.53% |
| GoogleNet | 3114.0 | 1849.4 | 40.61% |
| ResNet50 | 6326.6 | 4170.6 | 34.08% |

TABLE VIII
MULTIPLICATION SAVING RATE OF DIFFERENT ACCELERATION METHODS

| Output size | Kernel size | Stride | Multiplication and Saving rate[b] GEMM-based | FFT-based work [10] | FFA-based work [7] | This work |
|---|---|---|---|---|---|---|
| 2 | 3 | 1 | 36 | 22/38.9% | 24/33% | 16/55.6% |
| 4 | 5 | 1 | 400 | 94/76.5% | 240/40% | 256/36% |
| 2 | 5 | 2 | 100 | 94/6% | --[a] | 64/36% |
| 2 | 7 | 2 | 196 | -- | -- | 144/26.5% |
| 2 | 11 | 2 | 484 | -- | -- | 256/47.2% |

a. "--"means the work doesn't support this convolution shape.
b. The saving rate is compared to the baseline GEMM-based method.

### A. Performance Evaluation

The effectiveness of WHD fusing unit is accessed on the basis of how many multiplication operations can be saved compared to conventional convolution. Fig.15 shows the performance of WHD fusing unit for different convolution shapes. When stride $s = 1$, the multiplication saving rate fluctuates near 50% for the kernel size ranging from 2 to 16. As stride $s$ increases from 1 to 4, the multiplication saving rates drop rapidly for small kernel size. In most cases, WHD requires much less multiplications than conventional convolution. Exceptionally, in a few cases that stride $s$ is close to kernel size $r$ (e.g., $(r,s)=(3,2)$, $(r,s)=(4,3)$, and $(r,s)=(5,4)$), WHD provides insufficient yield, even sometimes negative saving rate. However, it should be noted that these convolution shapes seldom appear in current CNN models. For example, when $s = 4$ and $r = 5$, the multiplication reduction rate is $-14\%$, but this kind of convolution shape has lost much receptive fields during the inference.

As listed in TABLE VII, the effectiveness of operation reduction on WHD fusing unit is evaluated on six widely used CNN models. The number of operations includes convolutional layer and full connected layer. The best performance (i.e., 55.41% operation reduction) achieves on VGG16 among all these CNN models. This is because that VGG16 has only one convolution shape (i.e., $r = 3$ and $s = 1$), which fits Winograd *F(4,3)* well. Likewise, YOLOV1 [30] and DarkNet19 [31] also perform more than 50% operation saving rate on WHD fusing unit. Due to excessive zero padding incurred, the convolution shape with $1 \times 1$ kernel size degrades the operation saving rate on the fusing unit. The majority of convolutional layers in GoogleNet [32] and ResNet50 [17] employ $1 \times 1$ kernel size, so their operation saving rates are relatively low, which are 40.61% and 34.08% respectively. It is worth mentioning that these multiplications are indeed eliminated in the hardware implementation and don't occupy computation time. Because the output of transformation module in WHD fusing unit is set as $4 \times 4$, each PE in the fusing unit is running during the computation. In contrast, in some previous architectures [1], [10] the multiplication operations are saved, however, the multipliers still exist in PE and are merely inactive.

### B. Comparisons to Other Acceleration Algorithms

WHD fusing unit is an area-efficient implementation, along with SCDM to improve flexibility and extend usage range. Thus, we compare Winograd-based WHD fusing unit with FFT-based [10] and FFA-based [7] accelerators from four aspects of multiplication saving, transformation complexity, flexibility and hardware uniformity.

First, multiplication saving rates are compared. As shown in TABLE VIII, for most convolution shapes listed in this table, WHD fusing unit outperforms the other two competitors and achieves 26.5%∼55.6% multiplications reduction, compared to GEMM-based method. Specifically, for the most widely used $3 \times 3$ 1-stride convolution, WHD reduces 55.6% multiplications, which is much better than 33% reduction in FFA-based work [7] and 39% reduction in FFT-based work [10]. This advantage is conducive for WHD to achieve more performance improvements on current CNN models. However, there is one exception (i.e., kernel size is 5 and stride is 1), where WHD is inferior to FFT-based and FFA-based works. This is because WHD adopts small Winograd parameters, so that it needs to decompose the $5 \times 5$ kernel and fill with more zero paddings. In contrast, work [10] adopts 8 points FFT, which can well fit in this convolution shape (i.e., $r = 5$ and $s = 1$). However, 8 points FFT in work [10] is too small to accommodate 2-stride $5 \times 5$ kernel size, thereby performing worse than WHD. Due to FFT acceleration is unable to work under certain parameters, work [10] employs 8 sample point in its FFT unit design. As a result, when $(r + s - 1) > 7$, the FFT-based acceleration can't be deployed. As for the FFA-based work [7], it only has the FFA-based unit for kernel size 3/5/7 and stride 1, thus it can't support other more convolution shapes.

Second, the transformation complexity in pre-computation of WHD fusing unit is much simpler and more hardware-friendly, compared to FFA and FFT based designs. As shown in Fig.7 and Fig.8, the elements of transformation matrices in Winograd *F(4,3)* and *F(4,2)* are $\pm 1$, 0, and $\pm 0.5$, which are easily to be implemented on hardware and quantized with little precision loss. In contrast, the number of sample point used in the FFT-based acceleration unit [10] is 8, thereby the elements of its transformation matrices are $\pm 1$, 0, and $\pm 0.7071$. As listed in TABLE IX, if increasing the number of sample point to 16, the transformation matrices include much more complex elements, such as $\pm 1$, 0, $\pm 0.7071$, $\pm 0.9239$, and $\pm 0.3827$, which would cause more data precision loss. Besides, multiplication is also inevitable when accomplishing the pre-computation in FFT. As for FFA-based work [7], the transformation processes for different convolution shapes are non-identical and irregular. As the kernel size of convolution shape becomes large, the transformation in FFA becomes quite complex.

TABLE IX
TRANSFORMATION MATRICES ELEMENTS UNDER DIFFERENT NUMBER OF SAMPLE POINTS IN FFT

| # of sample point | Transformation Matrices Elements |
|---|---|
| 8 | ±1, 0, ±0.7071 |
| 16 | 0, ±0.3827, ±0.7071, ±0.9239, ±1 |
| 32 | 0, ±0.1951, ±0.3827, ±0.5556, ±0.7071, ±0.8315, ±0.9239, ±0.9808, ±1 |

TABLE X
FLEXIBILITY OF DIFFERENT ACCELERATION METHODS.

| | Kernel size r | Stride s |
|---|---|---|
| This work | Any | Any |
| FFA-based work [7] | 3/5/7 | 1 |
| FFT-based work [10] | 1-8 | 1 |

TABLE XI
THE HARDWARE OVERHEAD OF ONE WHD FUSING UNIT.

| | Input Transformation | Filter Transformation | Output Transformation | PE array | Accumulation | Total |
|---|---|---|---|---|---|---|
| ADD | 32 | 21 | 35 | 0 | 9 | 97 |
| SHIFT | 0 | 14 | 0 | 0 | 0 | 14 |
| MUX2 | 0 | 14 | 7 | 0 | 0 | 21 |
| INV | 16 | 7 | 7 | 0 | 0 | 30 |
| MULT | 0 | 0 | 0 | 16 | 0 | 16 |

Next, the difference in flexibility is evaluated. As shown in TABLE X, the WHD fusing unit can support any convolution shapes computation with the help of SCDM, which provides a highly flexibility and extendibility for rapidly evolving CNN models. In contrast, FFA-based acceleration work [7] only supports 1-stride with 3/5/7 kernel sizes, and FFT-based acceleration work [10] supports 1-stride with 1-8 kernel sizes.

Finally, hardware uniformity is discussed. Due to the high similarity in the transformation matrices and dot-product shapes of *F(4,3)* unit and *F(4,2)* unit fused in WHD, the fusing unit has a uniform computation architecture and achieves highly hardware reuse and fully logic sharing. Therefore, the transformation logics of *Tuniti, Tunitf,* and *Tunito* are simple and area-efficient, and the $4 \times 4$ computational PE array is neat and identical. Such effective architecture is conductive to enhance the utilization of PE and improve throughput. As a contrast, work [7] builds three types of FFA units and each of them required different shapes of multipliers' PE array. Similarly, Work [10] fuses Winograd unit and FFT unit together to accelerate different shapes. It causes that PE or multipliers cannot be fully reused for different convolution shapes, badly harms the hardware uniformity.

### C. Hardware Overhead

The SCDM method substitutes MULT of convolution with ADD and SHIFT. With the help of SCDM, WHD can flexibly process any convolution shapes and substantially reduce MULT at the cost of increasing INV, MUX2, ADD and SHIFT for $A/B/G$ matrices transformation. We evaluate the hardware cost of WHD fusing unit. As shown in TABLE XI, input transformation employs 32 ADDs and 16 INVs; filter transformation consumes 7 INVs, 21 ADDs, 14 SHIFTs and 14 MUX2s; output transformation needs 7 INVs, 35 ADDs and 7 MUX2s; accumulation includes 9 ADDs. The only part consuming multipliers is PE array which needs 16 multipliers. Thus, the hardware overhead of one fusing unit is 30 INVs, 97 ADDs, 16 MULTs, 14 SHIFTs and 21 MUX2s. It should be noted that SHIFT can be simplified into hard-wired connection of corresponding bits according to the elements in $A/B/G$ transformation matrices, thereby incurring no hardware cost.

TABLE XII
THE QUANTIZATION SCHEME OF WHD FUSING UNIT.

| Module | Signal | Quantization |
|---|---|---|
| Input Transformation | in | INT8 |
| | btin | INT9 |
| | btinb | INT10 |
| Filter Transformation | f | INT8 |
| | gf | INT9 |
| | gfgt | INT10 |
| Multipliers in PE array | input | INT10 |
| | output | INT20 |
| Output Transformation | uv | INT20 |
| | atuv | INT22 |
| | atuva | INT24 |
| Accumulation | input | INT24 |
| | output | INT25 |

TABLE XIII
THE COMPARISON OF HARDWARE RESOURCE AND POWER BETWEEN WHD FUSING UNIT AND GEMM UNIT

| | Resource | | Power(W) |
|---|---|---|---|
| | LUT | FF (bit) | |
| WHD | 2121 | 320 | 0.338 |
| GEMM | 2980 | 576 | 0.431 |

The WHD architecture was implemented and compared against conventional GEMM architecture. For the widely used $3 \times 3$ kernel size with 1 stride, one WHD fusing unit can process a $4 \times 4$ input tile and produce $2 \times 2$ output data via Winograd *F(4,3)* unit. To generate the same $2 \times 2$ size of output data, the competitor GEMM unit holds the same size of input tile and convolution shape with WHD fusing unit. In this case, the GEMM unit needs 36 multipliers to achieve the same throughput with WHD fusing unit. In this implementation, both of input data and filter weights of WHD are quantized as 8 bits fixed point data representation (i.e., INT8), which is widely used in many previous works [7], [40]. The detailed quantization scheme for WHD is listed in TABLE XII, where all the valid bits are reserved without any data truncation. Thus, the data precision is completely reserved. To maintain the same implementation precision, the input and output of the multipliers used in GEMM unit are INT8 and INT16, respectively.

To evaluate the hardware overhead and power consumption, both the WHD fusing unit and GEMM unit were synthesized using Vivado, where the device is Xilinx XC7Z100 FPGA and clock frequency is 200MHz. For direct and clear comparison,

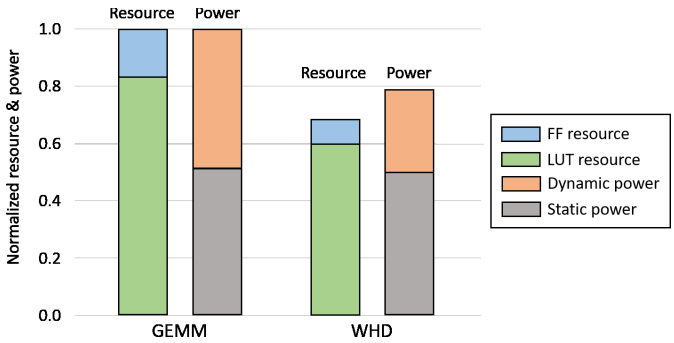



Fig. 16. Normalized comparison of hardware resource and power breakdown between WHD fusing unit and GEMM unit.

the multipliers used in WHD fusing unit and GEMM unit were both achieved by LUT instead of DSP. The synthesis results of WHD fusing unit and GEMM unit are listed in TABLE XIII. The on-chip buffers for filter weights, input feature maps and output results were not included in the synthesis results, because the capacity and bit width of these on-chip buffers can be flexibly customized according to memory bandwidth allocation and throughput requirement.

Normalized comparisons of hardware resource and power breakdown between WHD fusing unit and GEMM unit is depicted in Fig.16. Compared with the competitor GEMM unit, WHD fusing unit can save 28.8% LUT resource, 44.4% FF register, and 21.6% power consumption. In addition to the improvements on hardware resource and power, it is worth mentioning that the most important merit of WHD fusing unit with SCDM method is high flexibility, thereby supporting diverse convolution shapes on uniformed hardware architecture.

Systolic array (as in TPU [42]) and zero-skip (as in Eyeriss [41] and Cnvlutin [43]) are two typical hardware architectures implemented in current CNN accelerators. Although systolic array architecture achieves high performance by keeping all systolic PEs busy and simplifying the structure of PE (e.g., control logic and local buffer), this also leaves a large area requirement for on-chip global buffer. Besides, the processing states of systolic array architecture need to proceed in well-designed lock steps, which impose restrictions on flexibility for supporting various convolution shapes and induce data reorganization buffer for original matrices. As for the zero-skip architecture, skipping the zero-valued operations inevitably suffers from the irregularity, where irregular data access cannot be performed efficiently on hardware. Besides, extra zero detection logic and complex skipping control logic may cause an adverse effect on area efficiency. As a contrast, the PE array in WHD fusing unit processes data as concise and efficient as systolic array architecture, where each PE's control logic and local data register is completely eliminated from the PE array in WHD. Although input/filter/output transformations are involved in Winograd algorithm, no complex routing existed in the data path of WHD fusing unit because these transformations are performed by simple and less hardwired components. Therefore, compared with systolic array architecture and zero-skip architecture, WHD fusing unit does not introduce significantly degradation on area efficiency.

## V. Conclusion

This paper proposed an area-efficient CNN acceleration unit (i.e., WHD) that fuses Winograd *F(4,3)* and *F(4,2)* units by highly logic sharing and achieves complementary performance advantages. A novel stride-based convolution decomposition method (i.e., SCDM) was present to reform any convolution shapes to Winograd *F(4,3)* or *F(4,2)* pattern, which stretches Winograd for convolution shapes with large stride and kernel size, thereby achieving a highly flexible processing on WHD fusing unit. Evaluation results for AlexNet, VGG16, YOLOV1, DarkNet19, GoogleNet, and ResNet50 show that 34.08%$\sim$ 55.41% operation reduction were achieved on six CNN models, while incurring a slight hardware overhead. In future work, we intend to utilize an array of multiple WHD elements as the core component for a highly efficient and flexible CNN accelerator implementation.

## References

[1] L. Lu, Y. Liang, Q. Xiao, and S. Yan, "Evaluating fast algorithms for convolutional neural networks on FPGAs," in *Proc. IEEE 25th Annu. Int. Symp. Field-Program. Custom Comput. Mach. (FCCM)*, Apr./May 2017, pp. 101–108.

[2] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 4013–4021.

[3] R. DiCecco, G. Lacey, J. Vasiljevic, P. Chow, G. Taylor, and S. Areibi, "Caffeinated FPGAs: FPGA framework for convolutional neural networks," in *Proc. Int. Conf. Field-Program. Technol. (FPT)*, Dec. 2016, pp. 1–4.

[4] A. Podili, C. Zhang, and V. Prasanna, "Fast and efficient implementation of convolutional neural networks on FPGA," in *Proc. IEEE 28th Int. Conf. Appl.-Specific Syst., Archit. Processors (ASAP)*, Jul. 2017, pp. 11–18.

[5] L. Lu and Y. Liang, "SpWA: An efficient sparse Winograd convolutional neural networks accelerator on FPGAs," in *Proc. 55th ACM/ESDA/IEEE Design Autom. Conf. (DAC)*, Jun. 2018, pp. 1–6.

[6] D. A. Parker and K. K. Parhi, "Area-efficient parallel FIR digital filter implementations," in *Proc. Int. Conf. Appl. Specific Syst., Archit. Processors (ASAP)*, Aug. 1996, pp. 93–111.

[7] J. Wang, J. Lin, and Z. Wang, "Efficient hardware architectures for deep convolutional neural network," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 6, pp. 1941–1953, Jun. 2018.

[8] J. Yu *et al.*, "Instruction driven cross-layer CNN accelerator with Winograd transformation on FPGA," in *Proc. Int. Conf. Field Program. Technol. (ICFPT)*, Dec. 2017, pp. 227–230.

[9] C. Yang, Y. Wang, X. Wang, and L. Geng, "A reconfigurable accelerator based on fast Winograd algorithm for convolutional neural network in Internet of Things," in *Proc. IEEE ICSICT*, Oct./Nov. 2018, pp. 1–3.

[10] L. Lu, Y. Liang, Q. Xiao, and S. Yan, "Evaluating fast algorithms for convolutional neural networks on FPGAs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 4, pp. 857–870, Apr. 2020.

[11] J. Song *et al.*, "An 11.5TOPS/W 1024-MAC butterfly structure dual-core sparsity-aware neural processing unit in 8nm flagship mobile SoC," in *IEEE ISSCC Dig. Tech. Papers*, Feb. 2019, pp. 130–131.

[12] S. Yin *et al.*, "An ultra-high energy-efficient reconfigurable processor for deep neural networks with binary/ternary weights in 28 nm CMOS," in *Proc. IEEE Symp. VLSI Circuits*, Jun. 2018, pp. 37–38.

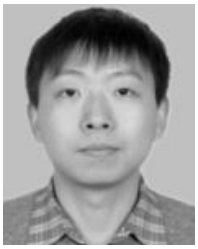
[13] T. Abtahi, C. Shea, A. Kulkarni, and T. Mohsenin, "Accelerating convolutional neural network with FFT on embedded hardware," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 26, no. 9, pp. 1737–1749, Sep. 2018.

[14] M. Mathieu, M. Henaff, and Y. Lecun, "Fast training of convolutional networks through FFTs," in *Proc. ICLR*, 2014, pp. 1–9. [Online]. Available: https://arxiv.org/abs/1312.5851

[15] M. Hemnani, S. Palekar, P. Dixit, and P. Joshi, "Hardware optimization of complex multiplication scheme for DSP application," in *Proc. Int. Conf. Comput., Commun. Control (IC)*, Sep. 2015, pp. 1–4.

[16] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.

[17] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
[18] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2014, pp. 580–587.
[19] D.-N. Ta, W.-C. Chen, N. Gelfand, and K. Pulli, "SURFTrac: Efficient tracking and continuous object recognition using local feature descriptors," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2009, pp. 2937–2944.
[20] J. R. R. Uijlings, K. E. A. van de Sande, T. Gevers, and A. W. M. Smeulders, "Selective search for object recognition," *Int. J. Comput. Vis.*, vol. 104, no. 2, pp. 154–171, Sep. 2013.
[21] P. Arbelaez, B. Hariharan, C. Gu, S. Gupta, L. Bourdev, and J. Malik, "Semantic segmentation using regions and parts," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2012, pp. 3378–3385.
[22] H. Noh, S. Hong, and B. Han, "Learning deconvolution network for semantic segmentation," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1520–1528.
[23] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*. [Online]. Available: http://arxiv.org/abs/1704.04861
[24] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4510–4520.
[25] X. Zhang, X. Zhou, M. Lin, and J. Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," 2017, *arXiv:1707.01083*. [Online]. Available: http://arxiv.org/abs/1707.01083
[26] J. Yue *et al.*, "A 65 nm 0.39-to-140.3TOPS/W 1-to-12b unified neural network processor using block-circulant-enabled transpose-domain acceleration with 8.1× higher TOPS/mm$^2$ and 6T HBST-TRAM-based 2D data-reuse architecture," in *IEEE ISSCC Dig. Tech. Papers*, Feb. 2019, pp. 138–139.
[27] S. Zhang *et al.*, "Cambricon-X: An accelerator for sparse neural networks," in *Proc. 49th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2016, pp. 1–12.
[28] A. Parashar *et al.*, "SCNN: An accelerator for compressed-sparse convolutional neural networks," in *Proc. ACM/IEEE 44th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2017, pp. 27–40.
[29] J. Lee, C. Kim, S. Kang, D. Shin, S. Kim, and H.-J. Yoo, "UNPU: A 50.6TOPS/W unified deep neural network accelerator with 1b-to-16b fully-variable weight bit-precision," in *IEEE ISSCC Dig. Tech. Papers*, Feb. 2018, pp. 218–219.
[30] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," 2015, *arXiv:1506.02640*. [Online]. Available: http://arxiv.org/abs/1506.02640
[31] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 6517–6525.
[32] C. Szegedy *et al.*, "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 1–9.
[33] C. Zhuge, X. Liu, X. Zhang, S. Gummadi, J. Xiong, and D. Chen, "Face recognition with hybrid efficient convolution algorithms on FPGAs," in *Proc. Great Lakes Symp. VLSI (GLSVLSI)*, 2018, pp. 123–128.
[34] S. Kala, B. R. Jose, J. Mathew, and S. Nalesh, "High-performance CNN accelerator on FPGA using unified Winograd-GEMM architecture," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 12, pp. 2816–2828, 2019.
[35] Q. Xiao, Y. Liang, L. Lu, S. Yan, and Y.-W. Tai, "Exploring heterogeneous algorithms for accelerating deep convolutional neural networks on FPGAs," in *Proc. 54th Annu. Design Autom. Conf. (DAC)*, 2017, pp. 1–6.
[36] H. Zeng, R. Chen, C. Zhang, and V. Prasanna, "A framework for generating high throughput CNN implementations on FPGAs," in *Proc. ACM/SIGDA FPGA*, 2018, pp. 117–126.
[37] C. Peng, X. Zhang, G. Yu, G. Luo, and J. Sun, "Large kernel matters—improve semantic segmentation by global convolutional network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 4353–4361.
[38] P. F. Schultz, D. M. Paiton, W. Lu, and G. T. Kenyon, "Replicating kernels with a short stride allows sparse reconstructions with fewer independent kernels," 2014, *arXiv:1406.4205*. [Online]. Available: http://arxiv.org/abs/1406.4205
[39] J. Wu, C.-W. Xie, and J.-H. Luo, "Dense CNN learning with equivalent mappings," 2016, *arXiv:1605.07251*. [Online]. Available: http://arxiv.org/abs/1605.07251
[40] S. Yin *et al.*, "A high energy efficient reconfigurable hybrid neural network processor for deep learning applications," *IEEE J. Solid-State Circuits*, vol. 53, no. 4, pp. 968–982, Apr. 2018.
[41] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
[42] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proc. ACM/IEEE 44th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2017, pp. 1–12.
[43] J. Albericio, P. Judd, T. Hetherington, T. Aamodt, N. E. Jerger, and A. Moshovos, "Cnvlutin: Ineffectual-neuron-free deep neural network computing," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2016, pp. 1–13.



**Chen Yang** received the B.S. degree in electronic engineering and the M.S. and Ph.D. degrees from Tsinghua University, Beijing, China, in 2004, 2007, and 2016, respectively. He was with VIA Technologies, Inc., Beijing, from 2007 to 2009. He is currently with the School of Microelectronics, Xi'an Jiaotong University, as a Lecturer. His current research interests include on-chip memory management technology, reconfigurable computing, and VLSI SoC design.

**Yizhou Wang** received the B.S. degree in electronic engineering from Xi'an Jiaotong University, Xi'an, China, in 2017, where he is currently pursuing the M.S. degree with the School of Microelectronics. His current research interests include on-chip memory management technology and reconfigurable computing.

**Xiaoli Wang** received the M.S. degree in electronic science and technology from Xi'an Jiaotong University. He has worked at the University of North Carolina at Charlotte and the University of Texas at Dallas as a Visiting Professor. His research interests are VLSI design, SoC design, ASIC design, and circuits simulation.

**Li Geng** received the B.Sc. degree in physics and the M.Sc. and Ph.D. degrees in electrical engineering from the Xi'an University of Technology, Xi'an, China, in 1990, 1998, and 2001, respectively. She was the Director of the School of Microelectronics, Xi'an Jiaotong University. She was a TPC member of ASSCC from 2010 to 2018. Her current research interests include power management integrated circuits, low-voltage low-power analog and mixed-signal SoC design, RF integrated circuit, and bioimplant systems.